

Modul 1: MySQL – Data Manipulation Language (DML) - INSERT

Topik

Penggunaan SQL Statement INSERT pada DBMS MySQL.

Tujuan

Setelah mempelajari modul ini, mahasiswa diharapkan dapat:

1. Memahami penggunaan SQL statement INSERT.
2. Menambahkan data ke dalam tabel menggunakan berbagai format INSERT.

Pendahuluan

DML merupakan istilah untuk beberapa sintaksis (syntax) dari SQL yang digunakan untuk melakukan perubahan pada data (isi tabel-tabel) dalam suatu database. DML terdiri dari 3 klausa utama yaitu:

1. **INSERT** : Menambah baris baru pada sebuah tabel
2. **UPDATE** : Mengubah nilai suatu baris pada sebuah tabel.
3. **DELETE** : Menghapus suatu baris dari sebuah tabel.

Pada DML terdapat dua jenis bahasa, yaitu :

1. High-Level(Non_procedural) DML.

- Digunakan secara interaktif (interpreter)
 - Dapat dijadikan satu dengan general purpose programming language (embedded)
- High-Level DML yang biasa digunakan secara interaktif disebut "Query Language".

2. Low-Level(Proedural) DML.

Digunakan secara embedded dalam suatu general purpose programming language

Bilamana kedua jenis DML diatas digunakan secara "embedded", maka : bahasa pemrograman yang digunakan disebut sebagai "Host Language" dan "DML-nya disebut "Sub Language"

Operasi INSERT

Operasi insert bertujuan untuk menyisipkan satu tuple baru ke dalam suatu relasi R.

Klausa pembentuk:

1. **INSERT**
2. **INTO**
3. **VALUES**

Format:

1. **INSERT INTO** nama_tabel (kolom1, kolom2, ...dst.) **VALUES** (nilai_kolom1, nilai_kolom2, ...dst.);
2. **INSERT INTO** nama_tabel **VALUES** (nilai_kolom1, nilai_kolom2, ...dst.); 3. [Salah satu dari kedua format sebelumnya], (nilai_kolom_kolom_baris1), (nilai_kolom_kolom_baris2), ...dst.

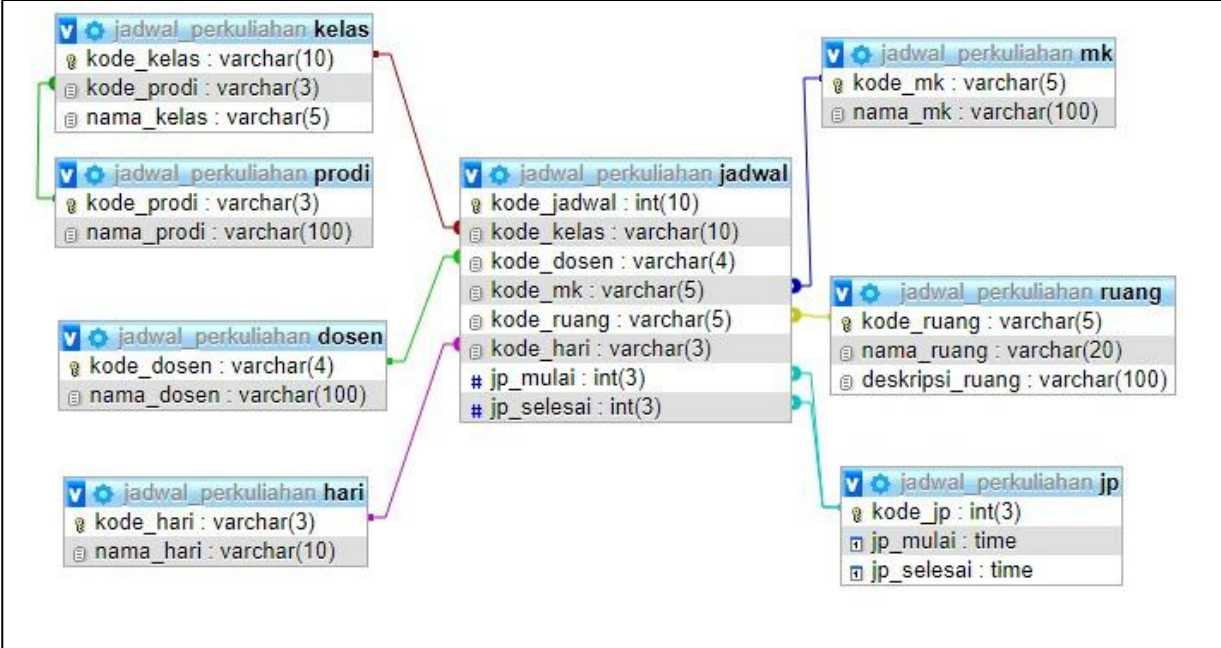
Operasi ini memungkinkan untuk melanggar empat jenis constraint sebagaimana dijelaskan berikut ini :

1. **DOMAIN Constraint** dapat dilanggar jika suatu nilai attribute yang diberikan tidak ada dalam domain yang berkorespondensi dengan attribute tadi.
2. **KEY Constraint** dapat dilanggar jika nilai key dalam tuple baru t sudah ada dalam tuple lain dalam relasi $r(R)$.
3. **ENTITY INTEGRITY** Constraint dapat dilanggar jika primary key dari tuple baru t adalah NULL
4. **REFERENTIAL INTEGRITY** Constraint dapat dilanggar jika nilai dari suatu foreign key dalam t mengacu ke suatu tuple yang tidak ada dalam relasi yang diacu.

Ada dua pilihan tindakan yang dapat dilakukan jika ada satu atau lebih constraint yang dilanggar akibat operasi insert, yaitu :

1. Menolak (reject) operasi insertion. Biasanya DBMS memberikan penjelasan mengapa proses insertion ditolak.
2. Berusaha memperbaiki alasan penolakan proses insertion. Dimana insertion akan diterima jika user melakukan perubahan nilai-nilai attribute sehingga insertion diterima.

Praktikum – Bagian 1: Percobaan Statement INSERT

Langkah	Keterangan
	Perhatikan skema/model relasional/EER diagram dari database berikut.  <pre> graph LR subgraph "jadwal_perkuliahan_kelas" k1[kode_kelas : varchar(10)] k2[kode_prodi : varchar(3)] k3[nama_kelas : varchar(5)] end subgraph "jadwal_perkuliahan_prodi" p1[kode_prodi : varchar(3)] p2[nama_prodi : varchar(100)] end subgraph "jadwal_perkuliahan_dosen" d1[kode_dosen : varchar(4)] d2[nama_dosen : varchar(100)] end subgraph "jadwal_perkuliahan_hari" h1[kode_hari : varchar(3)] h2[nama_hari : varchar(10)] end subgraph "jadwal_perkuliahan_jadwal" j1[kode_jadwal : int(10)] j2[kode_kelas : varchar(10)] j3[kode_dosen : varchar(4)] j4[kode_mk : varchar(5)] j5[kode_ruang : varchar(5)] j6[kode_hari : varchar(3)] j7[jp_mulai : int(3)] j8[jp_selesai : int(3)] end subgraph "jadwal_perkuliahan_mk" m1[kode_mk : varchar(5)] m2[nama_mk : varchar(100)] end subgraph "jadwal_perkuliahan_ruang" r1[kode_ruang : varchar(5)] r2[nama_ruang : varchar(20)] r3[deskripsi_ruang : varchar(100)] end subgraph "jadwal_perkuliahan_jp" jp1[kode_jp : int(3)] jp2[jp_mulai : time] jp3[jp_selesai : time] end k1 --> j2 k2 --> j2 p1 --> j2 d1 --> j3 d2 --> j3 h1 --> j6 h2 --> j6 j4 --> m1 j5 --> r1 j7 --> jp1 j8 --> jp1 </pre>
1.	Untuk menambahkan data (mengisi) suatu tabel, digunakan statement (pernyataan) INSERT. Eksekusi SQL berikut untuk menambahkan 1 baris (record) baru pada tabel mk. <pre>INSERT INTO mk (kode_mk, nama_mk) VALUES ('02010', 'Basis Data');</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang dinyatakan di dalam tanda kurung () pertama. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. Pembahasan lebih lengkap mengenai SELECT dijadwalkan untuk disampaikan pada pertemuan berikutnya, namun secara umum, statement SELECT digunakan untuk menyajikan record yang ada pada suatu tabel. Karakter * akan menampilkan isi dari semua kolom yang ada pada tabel. <pre>SELECT * FROM mk</pre>

2.	Apabila data di-insert-kan pada semua kolom tabel, maka kita dapat langsung menggunakan klausa VALUES tanpa harus menuliskan nama-nama kolom dahulu. <pre>INSERT INTO mk VALUES('02041', 'Teknologi Data');</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk tanpa menyebutkan nama kolomnya. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
3.	Untuk menambahkan beberapa kolom sekaligus dalam 1 statement digunakan statement dengan format seperti berikut. <pre>INSERT INTO mk VALUES (`02004`, `Aljabar Linier`), (`02005`, `Analisis Dan Desain Berorientasi Objek`), (`02006`, `Bahasa Indonesia`);</pre> Statement SQL tersebut menambahkan 3 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk tanpa menyebutkan nama kolomnya. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
4.	Dan seperti berikut, jika hanya kolom tertentu saja yang akan diberi nilai dengan cara menyebutkan nama kolomnya. <pre>INSERT INTO mk (kode_mk, nama_mk) VALUES (`02001`, `Agama`), (`02002`, `Aljabar Linier`), (`02003`, `Algoritma dan Struktur Data`);</pre>

	Statement SQL tersebut menambahkan 3 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
5.	Statement INSERT juga dapat dieksekusi dengan menggunakan klausa SET alih-alih VALUES. <pre>INSERT INTO mk SET kode_mk = `02012`, nama_mk = `Digital Entrepreneurship`;</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
6.	Pada statement INSERT juga dapat digunakan klausa SELECT. Misalnya kita ingin menyalin semua baris pada tabel mk ke tabel mk_backup, maka kita dapat menggunakan SQL berikut. (Buat terlebih dahulu tabel “mk_backup” dengan struktur tabel yang sama dengan tabel “mk”) <pre>CREATE TABLE mk_backup (kode_mk VARCHAR(5), nama_mk VARCHAR(100));</pre>
7.	Kemudian masukkan data dari tabel mk ke tabel mk_backup <pre>INSERT INTO mk_backup SELECT * FROM mk;</pre>
8.	Setelah berhasil mengeksekusi SQL tersebut, lanjutkan ke SubTopik selanjutnya.